



Classificador Bayesiano Simples
Introdução ao Reconhecimento de Padrões
Turma TB

Gabriel Vaz Cançado Ferreira
Matrícula: 2021015194

1. Implementação da densidade gaussiana bivariada independente

Inicialmente, cria-se uma função para calcular a densidade gaussiana de variáveis aleatórias unidimensionais, dada por:



```
def fnormal1var(mu, sigma, x):  
    return (1/(np.sqrt(2*np.pi)*sigma)) * np.exp(-((x-mu)/sigma)**2)
```

A partir disso, podemos usar essa função para a criação de uma função que calcule a densidade gaussiana bivariada independente. Assumindo que as variáveis são independentes podemos simplesmente computar o produto entre duas densidades gaussianas univariadas. Logo, a função criada é dada por:



```
def fnormal2var(m1, m2, s1, s2, x1, x2):  
    fNx1 = np.asmatrix(fnormal1var(m1, s1, x1))  
    fNx2 = np.asmatrix(fnormal1var(m2, s2, x2))  
    pxy = fNx1 * fNx2  
    return pxy
```

2. Geração dos dados

Geração de duas classes centradas nos pontos (m1x,m1y) e (m2x,m2y) e adição de ruído gaussiano:



```
xc1 = np.random.normal(loc=0, scale=0.8, size=(N,2)) + mean1  
xc2 = np.random.normal(loc=0, scale=0.8, size=(N,2)) + mean2
```

3. Visualização inicial das classes e das curvas de densidade das distribuições

Em seguida plotamos os dados e as curvas de densidades com base no grid criado:



```
xc1x, xc1y = zip(*xc1)
xc2x, xc2y = zip(*xc2)

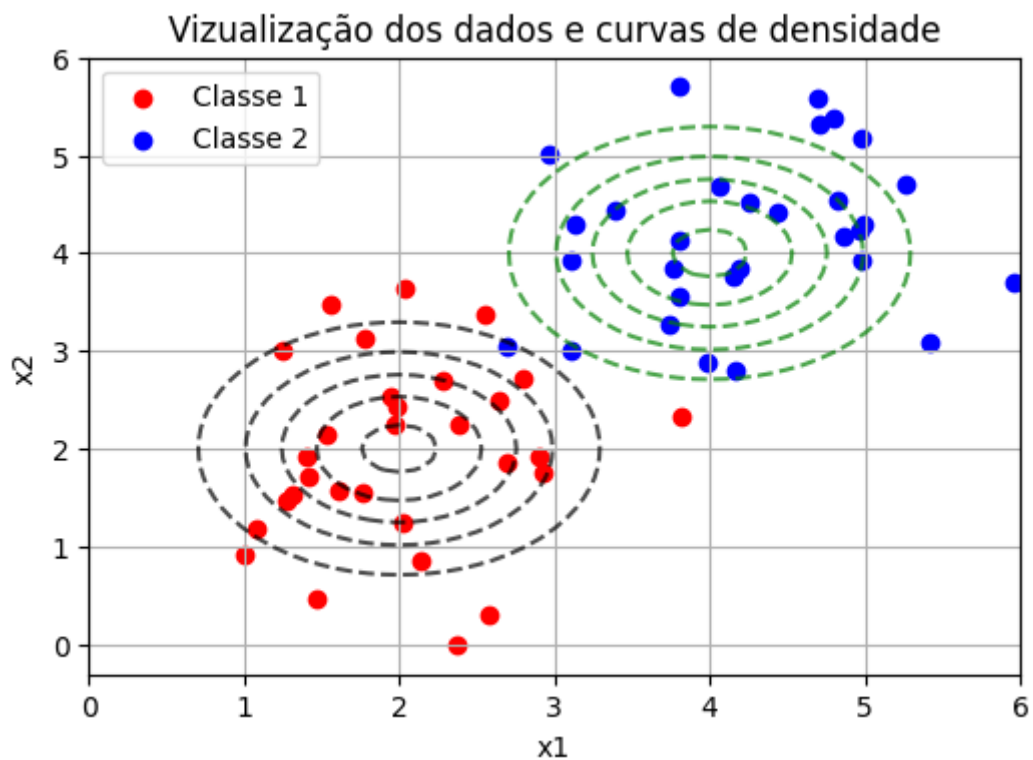
plt.figure(figsize=(6,4))
plt.scatter(xc1x, xc1y, color='red', marker='o', label='Classe 1')
plt.scatter(xc2x, xc2y, color='blue', marker='o', label='Classe 2')

seqx1x2 = np.arange(0, 6.1, 0.1)
lseq = len(seqx1x2)

MZ1 = np.empty((lseq, lseq))
MZ2 = np.empty((lseq, lseq))
diff = np.zeros((lseq, lseq))

for i in range(lseq):
    for j in range(lseq):
        x1 = seqx1x2[i]
        x2 = seqx1x2[j]
        MZ1[i, j] = fnormal2var(mean1[0], mean1[1], std1[0], std1[1], x1, x2).item()
        MZ2[i, j] = fnormal2var(mean2[0], mean2[1], std2[0], std2[1], x1, x2).item()
        diff[i, j] = MZ1[i, j] - MZ2[i, j]

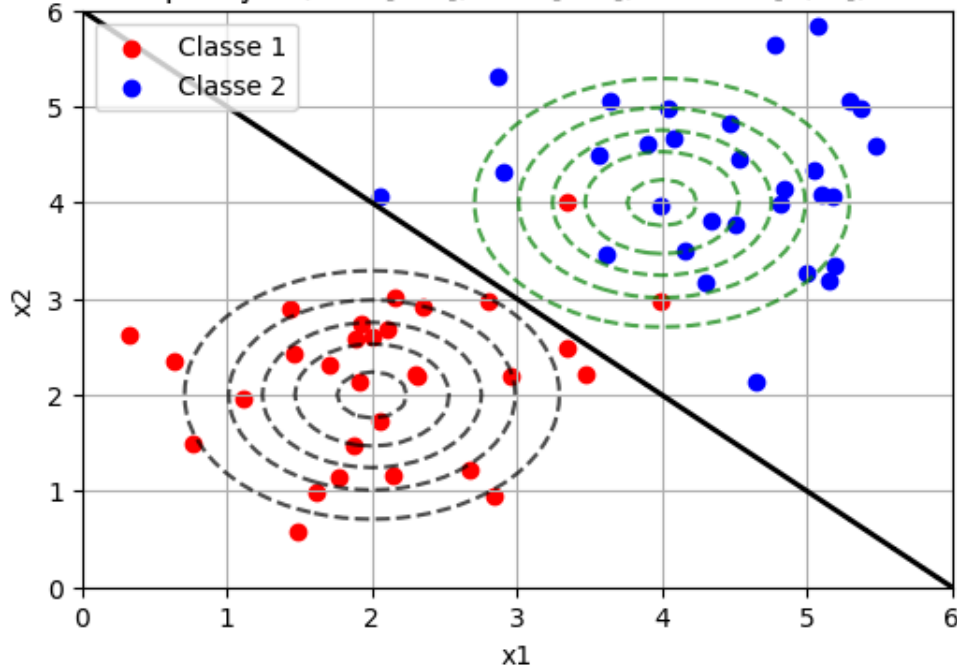
X, Y = np.meshgrid(seqx1x2, seqx1x2)
plt.contour(X, Y, MZ1.T, levels=5, colors='black', linestyle='dashed', alpha=0.7)
plt.contour(X, Y, MZ2.T, levels=5, colors='green', linestyle='dashed', alpha=0.7)
```



4. Classificação bayesiana simples

Por fim, plotamos a curva de separação. Para o caso em que possuímos médias diferentes e desvios padrão diferentes a curva de separação deverá ser a reta $x_2 = x_1$.

Fronteira de separação ($M1=[2\ 2]$, $M2=[4\ 4]$, $STD1 = [1, 1]$, $STD2 = [1, 1]$)



5. Experimentos

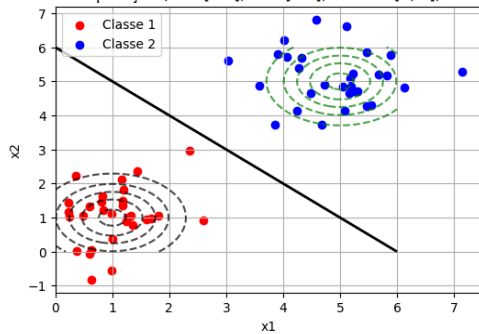
5.1 Variação das médias

Geramos os seguintes cenários de forma a ter um exemplo de problema com classes bem separadas e outro com superposição:

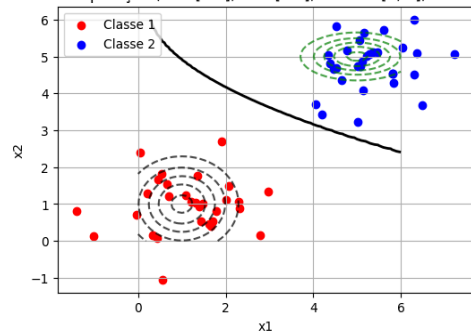
```
cenarios = [  
    # Classes bem separadas (médias distantes)  
    {"mean1": np.array([1, 1]), "mean2": np.array([5, 5]), "std1": [1, 1], "std2": [1, 1]},  
    {"mean1": np.array([1, 1]), "mean2": np.array([5, 5]), "std1": [1, 1], "std2": [1, 0.5]},  
    # Classes parcialmente sobrepostas  
    {"mean1": np.array([2, 2]), "mean2": np.array([3, 3]), "std1": [1, 1], "std2": [1, 1]},  
    {"mean1": np.array([2, 2]), "mean2": np.array([3, 3]), "std1": [1, 1], "std2": [1, 0.5]},  
    # Classes muito próximas (quase indistinguíveis)  
    {"mean1": np.array([2, 2]), "mean2": np.array([2.5, 2.5]), "std1": [1, 1], "std2": [1, 1]},  
    {"mean1": np.array([3, 2]), "mean2": np.array([3.2, 2.2]), "std1": [1, 1], "std2": [1, 0.5]}  
]
```

Os resultados obtidos podem ser observados a seguir:

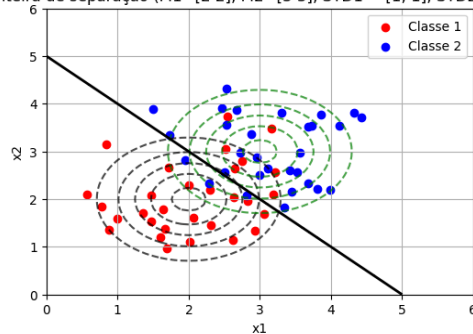
Fronteira de separação ($M1=[1 \ 1]$, $M2=[5 \ 5]$, $STD1 = [1, 1]$, $STD2 = [1, 1]$)



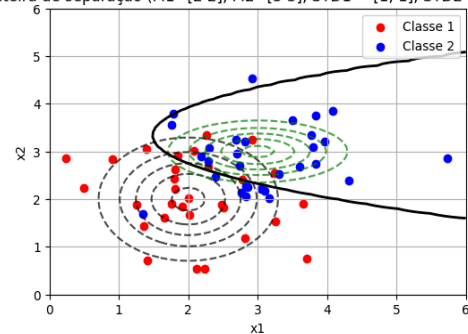
Fronteira de separação ($M1=[1 \ 1]$, $M2=[5 \ 5]$, $STD1 = [1, 1]$, $STD2 = [1, 0.5]$)



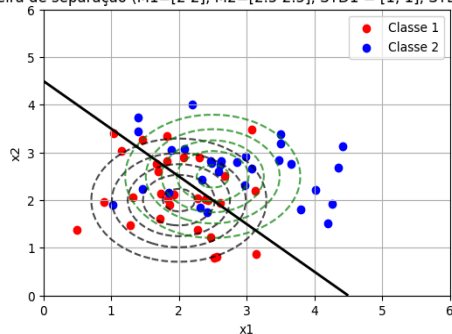
Fronteira de separação ($M1=[2 \ 2]$, $M2=[3 \ 3]$, $STD1 = [1, 1]$, $STD2 = [1, 1]$)



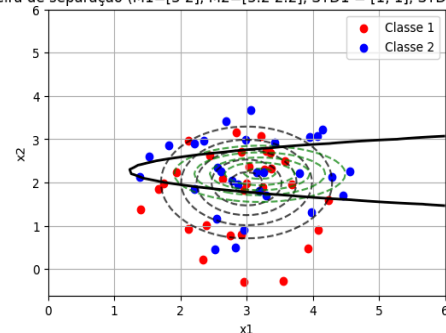
Fronteira de separação ($M1=[2 \ 2]$, $M2=[3 \ 3]$, $STD1 = [1, 1]$, $STD2 = [1, 0.5]$)



Fronteira de separação ($M1=[2 \ 2]$, $M2=[2.5 \ 2.5]$, $STD1 = [1, 1]$, $STD2 = [1, 1]$)



Fronteira de separação ($M1=[3 \ 2]$, $M2=[3.2 \ 2.2]$, $STD1 = [1, 1]$, $STD2 = [1, 0.5]$)



Notamos que em ambos os casos - dados com mesmo desvio padrão e diferente desvio padrão- a aproximação das médias gera claro prejuízo à qualidade da fronteira de separação, ao passo que médias muito distantes levam o modelo ao underfitting.

5.2. Variação dos parâmetros das distribuições normais bivariadas usadas para implementar o classificador

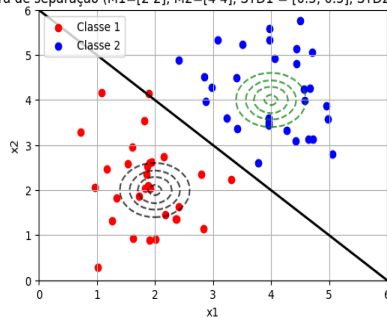
Geramos os seguintes cenários de forma a ter um exemplo de problema em que mantemos as médias fixas e variamos os desvios padrão em cada eixo (x,y) para ver o efeito da dispersão:



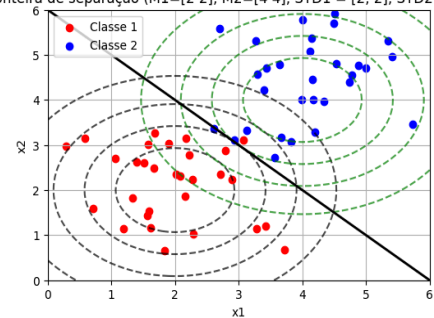
```
cenarios = [  
    # Ambas as classes compactas  
    {"mean1": np.array([2, 2]), "mean2": np.array([4, 4]), "std1": [0.5, 0.5], "std2": [0.5, 0.5]},  
  
    # Ambas as classes espalhadas igualmente  
    {"mean1": np.array([2, 2]), "mean2": np.array([4, 4]), "std1": [2, 2], "std2": [2, 2]},  
  
    # Uma classe compacta e outra espalhada  
    {"mean1": np.array([2, 2]), "mean2": np.array([4, 4]), "std1": [0.5, 0.5], "std2": [2, 2]},  
  
    # Dispersão maior em x do que em y  
    {"mean1": np.array([2, 2]), "mean2": np.array([4, 4]), "std1": [2, 0.5], "std2": [2, 0.5]},  
  
    # Dispersão maior em y do que em x  
    {"mean1": np.array([2, 2]), "mean2": np.array([4, 4]), "std1": [0.5, 2], "std2": [0.5, 2]},  
  
    # Classes com elipses orientadas em direções diferentes  
    {"mean1": np.array([2, 2]), "mean2": np.array([4, 4]), "std1": [2, 0.5], "std2": [0.5, 2]}  
]
```

Os resultados obtidos podem ser observados a seguir:

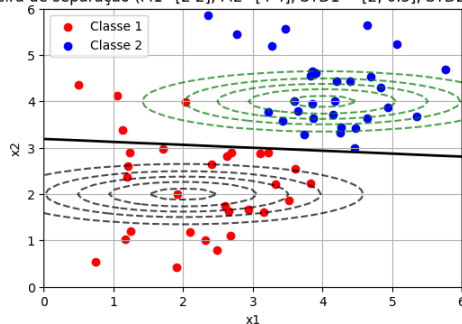
Fronteira de separação (M1=[2 2], M2=[4 4], STD1 = [0.5, 0.5], STD2 = [0.5, 0.5])



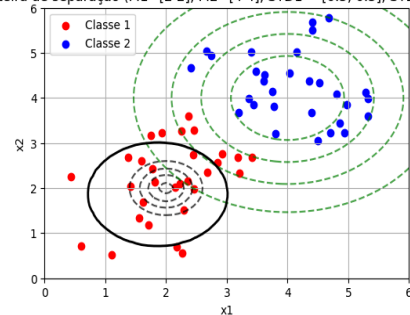
Fronteira de separação (M1=[2 2], M2=[4 4], STD1 = [2, 2], STD2 = [2, 2])



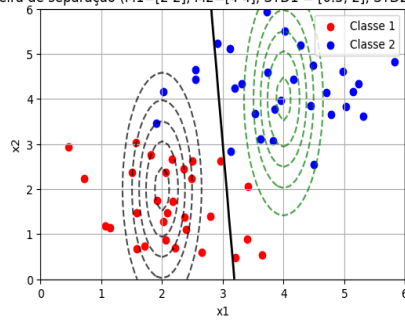
Fronteira de separação (M1=[2 2], M2=[4 4], STD1 = [2, 0.5], STD2 = [2, 0.5])



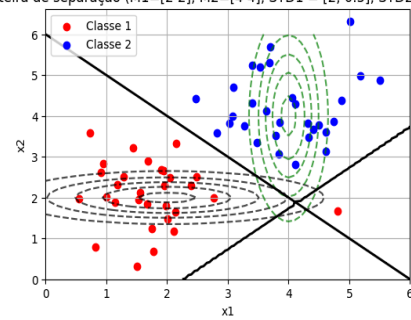
Fronteira de separação (M1=[2 2], M2=[4 4], STD1 = [0.5, 0.5], STD2 = [2, 2])



Fronteira de separação ($M1=[2\ 2]$, $M2=[4\ 4]$, $STD1 = [0.5, 2]$, $STD2 = [0.5, 2]$)



Fronteira de separação ($M1=[2\ 2]$, $M2=[4\ 4]$, $STD1 = [2, 0.5]$, $STD2 = [0.5, 2]$)



Por fim, vemos que a variação assimétrica (coluna da direita), ou seja, diferenças entre os desvios padrão nos eixos x e y são bastante prejudiciais à classificação do modelo ao passo que diferenças simétricas (coluna da esquerda) ainda permitem uma fronteira de separação bem definida.

6. Conclusão

O Trabalho permitiu a compreensão do algoritmo e princípios de funcionamento do classificador bayesiano simples, contando com sua implementação em Python e experimentação com os dados de treinamento. Além disso, o trabalho demonstrou como a compreensão dos dados é crucial para a boa performance do modelo, principalmente no que diz respeito às médias e desvios padrão dos dados usados no treinamento do modelo.